Migração do Waze Places para Cloudflare Workers/Pages

Documento técnico de planejamento — junho 2026

Análise de como migrar o Waze Places do stack atual (PHP + Apache) para Cloudflare Pages + Workers, mantendo um **fallback pra VM RedHat**. O desenho segue o mesmo padrão que deu certo no projeto **botequei**: um núcleo de lógica compartilhado com adaptadores finos por plataforma.

> **Nota (implementado na v3.0):** este documento é o planejamento original e cita um adaptador estilo *Pages Functions* (`functions/api/*.js`). Na implementação final optou-se pelo modelo **Worker com static assets** (`worker/index.mjs` + `assets` no `wrangler.jsonc` , deploy `npx wrangler deploy`) — igual ao botequei. A lógica (`server/core.mjs`) e o resto do plano valem sem mudança; só o formato do adaptador Cloudflare difere do descrito aqui.

---

1. Ponto de partida: o que temos hoje

Camada	Hoje
Frontend	HTML + JS vanilla + Tailwind (bundle local) + service worker
Backend	9 arquivos PHP em `api/` — proxies stateless pro Waze
Sessão	Blob AES-256-CBC em `/tmp/waze_places_sessions/` , chave em `/tmp/waze_places.key`
HTTP → Waze	cURL com cookies + headers (Referer/CSRF/Origin)
Servidor	Apache + `.htaccess` (rewrite, headers, cache, CSP, mod_pagespeed off)
Deploy	`git clone` + `restorecon` + `setsebool` na VM RedHat

O backend é pequeno e bem delimitado: ~600 linhas de PHP, quase tudo lógica pura + um punhado de chamadas HTTP. É justamente isso que torna a migração viável.

---

2. A lição do botequei: core compartilhado + adaptadores

O botequei roda **na mesma base de código** tanto no Cloudflare quanto numa VM Node.js. O truque:

```
server/core.mjs   → lógica pura (sem I/O de plataforma)
worker/index.mjs  → adaptador Cloudflare (Durable Objects, KV)
server/node.mjs   → adaptador VM (Node HTTP, filesystem)
```

O `core.mjs` não sabe onde está rodando. Os adaptadores injetam o que é específico de plataforma (armazenamento, secrets). **Mudou o protocolo? Atualiza os dois alvos de uma vez, porque a lógica é uma só.**

Isso mata a objeção clássica de "manter dois backends dobra o trabalho". Não dobra — o que duplica é só a casca fina de cada adaptador (algumas dezenas de linhas cada).

Como isso mapeia no Waze Places

```
server/core.mjs      → tudo que hoje é lógica em config.php + os 9 endpoints:
                       - URLs por região, extração de CSRF
                       - cripto AES (Web Crypto — roda em Worker E Node)
                       - callWaze() via fetch()
                       - categorizeWazeError, isUserAllowed
                       - parsers de buscar-places / perfil / listas
                       - abstração SessionStore (interface)

functions/api/*.js   → adaptador Cloudflare Pages Functions
                       (injeta SessionStore = Workers KV, chave = Secret)

server/node.mjs      → adaptador VM RedHat
                       (injeta SessionStore = filesystem, chave = env/arquivo)
```

Ponto importante: **`fetch()` e `crypto.subtle` (Web Crypto) existem nativamente tanto nos Workers quanto no Node 18+**. Ou seja, o `core.mjs` — incluindo a criptografia e as chamadas ao Waze — roda **idêntico** nos dois lugares. A única coisa que muda entre plataformas é onde as sessões são guardadas.

---

3. Mapa de conversão: PHP → JS

Todo o `api/config.php` + endpoints viram JS. Item a item:

Hoje (PHP)	Vira	Onde	Observação
`wazeIssuesEndpoint` etc. (6 helpers de URL)	funções puras	core	1:1
`wazeRefererEnv`	função pura	core	1:1
`extractCSRFToken`	regex	core	1:1
`validateCookiesFormat`	função pura	core	1:1
`categorizeWazeError`	função pura	core	1:1 (a joia — parsing de `errorList[0].code`)
`isUserAllowed`	função pura	core	1:1 (gate: staff OU rank≥2 & AM)
`createTempCookieFile`	**removido**	—	`fetch` manda header `Cookie:` direto, sem arquivo temp
`makeCurlRequest` (7 usos de cURL)	`callWaze()` com `fetch()`	core	mais limpo; sem `-k`, sem cookie file
`openssl_encrypt` AES-256-CBC	`crypto.subtle` (Web Crypto)	core	recomendo subir pra **AES-GCM** (autenticado) na migração — ver §7
`getEncryptionKey` (arquivo / tmp)	Secret (CF) / env (VM)	adaptador	injetado no core
`createSession` / `loadSession` / `destroySession`	`SessionStore` interface	core + adaptador	KV no CF, filesystem na VM
`cleanExpiredSessions` + `filemtime` / `touch`	**removido no CF** (TTL nativo do KV)	—	na VM continua com varredura, ou usa mtime
`getServerWorkers` / `PHP_CLI_SERVER_WORKERS`	**removido**	—	Workers escalam por request; Node é async
`readJsonInput` / `jsonResponse` / `jsonError`	helpers de Request/ Response	adaptador	Web `Request` / `Response` API
Parsers dos 9 endpoints	funções puras `handleX(input) → output`	core	o grosso do trabalho de port

Os parsers (especialmente `buscar-places`: filtro de permissão bitmask, `extractLonLat` recursivo, `resolveIdField` de rua/cidade, um-card-por-PUR, pareamento de imagem `image.id === updateRequest.id`) são o que dá mais trabalho — mas é tradução direta de lógica que já está testada e documentada.

---

4. Frontend no Pages (a parte fácil)

`index.html`, `css/`, `js/`, `icons/`, `manifest.json`, `service-worker.js` vão pro Pages **sem tocar em nada**. Só precisam estar na raiz do projeto (ou numa pasta de output configurada).

Ganho colateral enorme: **todo o `.htaccess` some**, e com ele os gotchas mais dolorosos do projeto:

- `mod_pagespeed off` — não existe no CF
- Rocket Loader / Auto Minify quebrando ordem de script — não existe
- Toda a saga de `Cache-Control` de 3 camadas (gotcha #18) — vira um arquivo `_headers` de ~5 linhas
- Redirect HTTPS, HSTS — nativos do CF

Headers passam a viver num arquivo `_headers` na raiz:

```
/*
  X-Content-Type-Options: nosniff
  Referrer-Policy: strict-origin-when-cross-origin
  Content-Security-Policy: default-src 'self'; ...
/service-worker.js
  Cache-Control: no-cache, no-store, must-revalidate
/js/*
  Cache-Control: no-cache, must-revalidate
/css/*
  Cache-Control: no-cache, must-revalidate
```

---

5. Sessões: de `/tmp` para adaptável

A interface no core:

```
// core.mjs
export function makeSessionApi({ store, getKey }) {
  // store.get(hash) -> Promise<string|null>
  // store.put(hash, blob, ttlSeconds) -> Promise<void>
  // store.delete(hash) -> Promise<void>
  // getKey() -> Promise<CryptoKey>
  return { createSession, loadSession, destroySession };
}
```

Adaptador Cloudflare (Workers KV)

```
// functions/api/_store.js
export const kvStore = (env) => ({
  get: (h) => env.SESSIONS.get('sess_' + h),
  put: (h, blob, ttl) => env.SESSIONS.put('sess_' + h, blob, { expirationTtl: ttl }),
  delete: (h) => env.SESSIONS.delete('sess_' + h),
});
```

O KV tem **TTL nativo** — a sessão some sozinha após 21 dias. Isso apaga `cleanExpiredSessions()` inteira.

> **[Superado]** A sessão hoje é DESLIZANTE (gotcha #41) e o VALOR do cookie do Waze morre em ~24–26 h sem renovação; o «Manter conectado» renova pelo Cron (ver o CLAUDE.md). O raciocínio de «~28 dias» abaixo é histórico.
>
> **Ajuste de design importante:** hoje o PHP faz `touch()` a cada uso pra estender a sessão. No KV, cada `touch` seria uma escrita, e o free tier só dá **1.000 escritas/dia**. A solução é *não* renovar a cada uso: setar TTL de 21 dias no login e pronto. Como os cookies do Waze morrem em ~28 dias de qualquer forma, estender a sessão além disso não tem efeito prático — quando o cookie expira, o fluxo de 401 → tela de login já cuida. Menos escrita, mesmo comportamento.

Adaptador VM RedHat (filesystem)

```
// server/node.mjs
import { readFile, writeFile, unlink } from 'node:fs/promises';
const dir = process.env.SESSION_DIR || join(process.env.STATE_DIRECTORY ?? join(homedir(), '.waze-p
export const fsStore = {
  get: (h) => readFile(`${dir}/sess_${h}`, 'utf8').catch(() => null),
  put: (h, blob) => writeFile(`${dir}/sess_${h}`, blob, { mode: 0o600 }),
  delete: (h) => unlink(`${dir}/sess_${h}`).catch(() => {}),
};
```

Praticamente o mesmo modelo de hoje (`/tmp`), só que em Node. A VM não tem limite de escrita, então pode inclusive manter o `touch` se quiser.

Chave de criptografia

- **Cloudflare:** `wrangler pages secret put ENCRYPTION_KEY` (ou no dashboard). Fica criptografada, injetada como `env.ENCRYPTION_KEY`.
- **VM:** variável de ambiente no systemd, ou gerar num arquivo `0600` como hoje.

---

6. Estrutura de arquivos proposta

```
wazeplaces/
├── index.html, css/, js/, icons/, manifest.json, service-worker.js   (inalterados)
├── functions/                      # Cloudflare Pages Functions
│   └── api/
│       ├── buscar-places.js        # onRequestPost → core.handleBuscarPlaces
│       ├── marcar-lido.js
│       ├── validar-place.js
│       ├── perfil.js
│       ├── lista-paises.js
│       ├── lista-estados.js
│       └── sessao.js
├── server/
│   ├── core.mjs                    # TODA a lógica (compartilhado)
│   └── node.mjs                    # adaptador VM RedHat
├── _headers                        # headers/CSP do CF (substitui .htaccess)
├── _redirects                      # (se precisar)
├── wrangler.jsonc                  # config CF (binding do KV)
└── api/                            # PHP LEGADO — remover após validar CF+VM
```

Estratégia segura: **manter `api/` PHP até o CF e a VM Node estarem validados em produção**. Só então remover o PHP.

---

7. Detalhes técnicos que exigem atenção

Web Crypto (cripto): `openssl_encrypt('aes-256-cbc')` vira `crypto.subtle.encrypt`. Como a migração começa do zero (as sessões em `/tmp` são efêmeras, não há dado a preservar), **recomendo subir pra AES-GCM** em vez de CBC: é autenticado (detecta adulteração do blob), mesmo custo de código, e é o padrão moderno. O formato de armazenamento muda de `iv::ciphertext` pra `iv::ciphertext+tag` — irrelevante já que ninguém depende do formato antigo.

`fetch` ao Waze: os headers críticos (Referer, X-CSRF-Token, Origin, sec-ch-ua-*, sec-fetch-*) continuam iguais — só mudam de `curl_setopt` pra um objeto `headers` no `fetch`. O Waze não vê diferença.

KV é eventualmente consistente globalmente: uma escrita leva um instante pra propagar entre regiões. No nosso caso (sessão criada e usada logo em seguida pelo mesmo cliente, mesma região), na prática não dá problema — mas vale saber.

Local dev muda: hoje é `./start.sh` (PHP). No novo modelo: - Cloudflare: `npx wrangler pages dev` (precisa Node + wrangler) - VM Node: `node server/node.mjs`

Isso é uma **mudança no valor "roda em qualquer host PHP"** do projeto. A contrapartida é que Node é tão universal quanto PHP hoje, e o fallback VM (seção 8) mantém a opção de rodar num servidor próprio.

---

8. Fallback VM RedHat (o "plano B" fora do Cloudflare)

Espelhando o que o botequei documenta. Se um dia precisar sair do Cloudflare, a mesma base roda numa VM com Node:

```
sudo dnf install -y nodejs git
sudo git clone https://github.com/antigerme/wazeplaces /opt/wazeplaces
# chave de sessão (gere uma vez):
export ENCRYPTION_KEY=$(openssl rand -base64 32)
node /opt/wazeplaces/server/node.mjs
```

Serviço systemd: o exemplo completo e endurecido (usuário sem privilégios, `StateDirectory`, chave em `EnvironmentFile` só do root) está no README, seção «Opção B». As sessões e a chave NÃO ficam mais no `/tmp`: `STATE_DIRECTORY` (systemd) ou `~/.waze-places`.

HTTPS via Apache (reverse proxy na frente do Node — opção preferida do owner). `/etc/httpd/conf.d/wazeplaces.conf`:

```
<VirtualHost *:80>
    ServerName places.seudominio.com
    ProxyPreserveHost On
    ProxyPass        / http://127.0.0.1:8080/
    ProxyPassReverse / http://127.0.0.1:8080/
    RequestHeader set X-Forwarded-Proto "http"
</VirtualHost>
```

```
sudo setsebool -P httpd_can_network_connect 1     # SELinux: proxy → porta local
sudo systemctl restart httpd
sudo dnf install -y certbot python3-certbot-apache
sudo certbot --apache -d places.seudominio.com    # cria o vhost :443 + redirect
```

Como o backend agora é Node (não mais PHP via mod_php), o Apache aqui é **só reverse proxy** — quem serve estáticos + `/api/*` é o `server/node.mjs`. Único boolean SELinux necessário: `httpd_can_network_connect`. Alternativa nginx:

```
location / {
    proxy_pass http://127.0.0.1:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $remote_addr;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Vantagem sobre o Apache+PHP antigo: um processo Node só, sem `restorecon`, sem `PHP_CLI_SERVER_WORKERS`, e é **a mesma** `server/core.mjs` que roda no Cloudflare — zero divergência de comportamento.

---

9. Passo a passo da migração

1. **Extrair o core**: reescrever `api/config.php` + endpoints em `server/core.mjs` (funções puras + `callWaze` via fetch + `SessionStore` como interface injetada).
2. **Adaptador Node** (`server/node.mjs`): HTTP server servindo estáticos + roteando `/api/*` pro core, com `fsStore`. Testar local com `node`.
3. **Testar contra o Waze real** (você — sandbox e agente não alcançam waze.com). Validar login, buscar, rejeitar, marcar lido com um HAR de referência.
4. **Adaptador Cloudflare**: `functions/api/*.js` chamando o mesmo core com `kvStore`. Criar namespace KV, setar Secret `ENCRYPTION_KEY`.
5. `_headers` com CSP traduzida; `wrangler.jsonc` com binding do KV.
6. **Deploy Pages** conectado ao GitHub (deploy automático no push pra `main`).
7. **Validar CF em produção** com um subconjunto de usuários.
8. **Remover** `api/` **PHP** e o `.htaccess` só depois de CF + VM validados.
9. **README**: documentar os dois caminhos de deploy (Cloudflare e VM RedHat), como o botequei faz.

---

10. Esforço e custo

Item	Esforço
Frontend → Pages	~2h (quase tudo teste)
Core em JS (`core.mjs`)	1-2 dias (port dos 9 endpoints + cripto)
Adaptador Node (VM)	~3h
Adaptador Cloudflare + KV + Secret + `_headers`	~3h
Testes contra Waze real	~meio dia (iterativo, depende de você)
Total	**~3-4 dias** de trabalho focado

Free tier Cloudflare: - Pages Functions / Workers: 100.000 requests/dia - KV: 100.000 leituras/dia, 1.000 escritas/dia, 1 GB

Cada chamada de API lê 1 sessão do KV. 100k leituras/dia ≈ 200 sessões pesadas (500 PURs cada). Login = 1 escrita, então 1.000 escritas/dia = 1.000 logins/dia — folgado. Se crescer além disso, o plano **Workers Paid ($5/mês)** dá 10M leituras/mês. A VM RedHat não tem nenhum desses limites (é seu servidor).

---

11. Recomendação

- **Frontend no Pages**: vale sozinho, risco quase zero, mata metade dos gotchas de cache/Apache. Dá pra fazer isso já, mesmo mantendo o backend PHP em outro host durante a transição (com um proxy `/api/*`, ou movendo backend junto).
- **Backend no core+adapters**: o padrão do botequei prova que não é difícil e não te prende ao Cloudflare — a VM RedHat continua sendo opção com a mesma base. Vale se você quer parar de manter Apache/RHEL/SELinux e ganhar escala automática + edge global.
- **Ordem sugerida**: extrair o core → validar na VM Node (mais fácil de testar, é seu ambiente) → só então plugar no Cloudflare. Assim você nunca fica sem um caminho que funciona.

O maior "custo" não é técnico — é aceitar que rodar local vira `node` em vez de `php -S`. Dado que o botequei já seguiu esse caminho sem dor e você gostou do resultado, o precedente é bom.

Documento gerado para planejamento e discussão com a equipe do Waze Places — junho 2026. Baseado no padrão de deploy validado no projeto botequei (core compartilhado + adaptadores Cloudflare/VM).